Guía del taller

Taller virtual ATA EPE · Cómo se estudia, se construye y se publica

Para qué es esta guía. El taller es un curso que se aprende construyendo: nueve módulos, cada uno con una ficha de estudio, unas pruebas y una función nueva en la web. Esta guía explica el ciclo de trabajo de cada módulo, las herramientas, las reglas del código y cómo se sube y se publica lo hecho.

Índice

1. **El mapa del curso** **2**
2. **El ciclo de cada módulo** **2**
3. **Herramientas** **3**
4. **Carpetas** **3**
5. **Reglas del código** **3**
6. **Git: ramas, commits y publicación** **3**
7. **Cómo trabajar con Claude en el taller** **4**
8. **La bitácora** **4**
9. **Checklist de cierre de cada módulo** **4**

1. El mapa del curso

	Ficha	Tema	Horas
0	`ficha00_pga`	Del producto geométrico a la PGA: el lenguaje del simulador	15
1	`ficha01_cinematica`	Movimientos rígidos, tornillos, velocidad, armar e imantar piezas	15
2	`ficha02_dinamica`	Masa, inercia, Newton–Euler en PGA, integradores en el grupo	20
3	`ficha03_analitica_i`	Ligaduras, grados de libertad, ejes, rango del jacobiano	15
4	`ficha04_ligaduras`	Uniones dinámicas, tornillos y tuercas, cuerpos compuestos	25
5	`ficha05_contacto`	Colisiones, contacto unilateral y fricción de Coulomb	35
6	`ficha06_transmisiones`	Ruedas de fricción, bandas elásticas, manivela	20
7	`ficha07_analitica_ii`	Lagrangiano reducido, trabajo virtual, «Ver las cuentas»	15
8	`ficha08_compartir`	Guardar, compartir, retos y realidad aumentada	10
		Total	**≈ 170 h**

Las horas son una estimación para alguien que estudia y programa a la vez. A un ritmo de 6 a 8 horas por semana, el curso completo toma de 5 a 7 meses. El orden importa: cada módulo usa lo de los anteriores.

2. El ciclo de cada módulo

Paso		Qué se hace
1	Leer	Las lecturas de la ficha, en el orden indicado. Lo marcado en negrita es lo esencial.
2	Hacer	Los ejercicios a mano, en papel o en `estudio/soluciones/fichaNN.tex`.
3	Responder	Las preguntas de control, por escrito. Si una no sale, se vuelve a las lecturas.
4	Discretizar	Escribir en pseudocódigo el algoritmo del módulo, con sus fuentes de error.
5	Contrato	Se generan las pruebas y el esqueleto del módulo (con Claude, a partir de la ficha y de lo ya implementado).
6	Implementar	En una rama propia, hasta que pasen las pruebas.
7	Verificar	Pruebas automáticas, prueba visual en el navegador y revisión del código.
8	Subir	Fusionar la rama con `main` y subir: GitHub Pages publica solo.
9	Cerrar	Marcar el módulo en la bitácora y anotar lo aprendido y lo pendiente.

La regla de oro

No se escribe código de un módulo sin haber hecho los pasos 1 a 4. Si al programar aparece algo que no se entiende, se vuelve a la ficha: el código es la última expresión de una idea que ya está clara en papel.

3. Herramientas

Herramienta	Para qué
Node 22	Correr las pruebas: `node --test "pruebas/*.test.js"` desde `taller/`.
Navegador	Ver la cara pública. Servidor local desde la carpeta `Cartilla`: `python3 -m http.server 8765`, y abrir `http://localhost:8765/taller/`.
Git y GitHub	Ramas por módulo, historial y publicación (el repositorio de la web).
LaTeX	Las fichas y las soluciones: `latexmk -pdf fichaNN_*.tex`.
Editor	Cualquiera con resaltado de JavaScript (VS Code, por ejemplo).

4. Carpetas

```
taller/
  plan/          el plan general (modelo físico, módulos, bibliografía)
  estudio/       esta guía, las fichas, comun.tex, soluciones/ y bitacora.md
  nucleo/        el motor: pga, cinemática, cuerpo, integrador, ligaduras, colisión, contacto
  kit/           lo propio del kit: catálogo con puertos, ensamble, tornillería, ruedas, bandas
  analisis/      grados de libertad, lagrangiano reducido, trabajo virtual
  app/           la cara pública: escena, interfaz, instrumentos
  pruebas/       una prueba por archivo del núcleo, del kit y del análisis
  index.html     la entrada pública del taller
```

5. Reglas del código

1. **Nombres en español**, como el resto de la web: `velocidad`, `motor`, `inercia`.
2. **Cada función dice qué ecuación implementa**: «ficha 2, ec. 3.4». Si no hay ecuación, falta teoría.
3. **El núcleo no conoce el dibujo.** `nucleo/`, `kit/` y `analisis/` no importan three.js ni tocan la página; así se prueban con Node. Solo `app/` dibuja.
4. **Sin librerías de física ni de álgebra.** Todo lo numérico se escribe aquí.
5. **Multivectores en `Float64Array(16)`**, como en la ficha 0. Las versiones rápidas (especializadas) se prueban siempre contra las generales.
6. **Unidades CGS** dentro del núcleo (cm, g, s); conversión a SI solo al mostrar.
7. **Determinismo**: mismo montaje, mismo paso de tiempo, mismo resultado. Nada de `Math.random` sin semilla en el núcleo.
8. **Pruebas primero**: cada error encontrado se convierte en una prueba nueva antes de corregirlo.

6. Git: ramas, commits y publicación

Cada módulo vive en su rama hasta que está cerrado:

```
git checkout main && git pull                  # partir de lo último
git checkout -b modulo-01-cinematica           # rama del módulo
# ... estudiar, implementar ...
node --test "pruebas/*.test.js"                # desde taller/: todo en verde
git add taller && git commit -m "Módulo 1: motor entre rectas e imantado"
git push -u origin modulo-01-cinematica        # copia de seguridad en GitHub
```

Al cerrar el módulo:

```
git checkout main && git merge modulo-01-cinematica
git push                                       # GitHub Pages publica /taller/ en 1-2 minutos
```

Mensajes de commit

En español y en presente, diciendo qué cambia: «Módulo 2: integrador semiimplícito en el grupo». Un commit por idea: es más fácil volver atrás y se lee como un diario del aprendizaje.

7. Cómo trabajar con Claude en el taller

- **Contrato del módulo** (paso 5): «genera el contrato del módulo 2». Claude escribe las pruebas y el esqueleto a partir de la ficha y de tu código de los módulos anteriores, y valida las pruebas contra una implementación de referencia que queda fuera del proyecto, en `~/Documents/Taller_referencia`.
- **Revisión**: «revisa mi `pga.js`». Claude explica cada corrección con la ecuación detrás, sin reescribir tu código salvo que lo pidas.
- **Dudas de teoría**: preguntar por la ecuación concreta de la ficha; si la ficha tiene un error, se corrige la ficha.
- **La referencia** existe para comparar *después* de intentarlo, no antes.

8. La bitácora

En `estudio/bitacora.md`, una entrada por sesión de trabajo:

```
## 2026-11-02 · Módulo 0 · 2 h
- Leído: Doran y Lasenby §2.1-2.4.
- Hecho: ejercicios 0.1 y 0.2. El signo del rotor: e1 gira hacia e2.
- Duda: por qué el regresivo necesita el dual de Poincaré.
- Siguiente: ejercicio 0.3 y productoBlades.
```

Sirve para retomar sin perder el hilo y, al final, es el registro de todo el proyecto.

9. Checklist de cierre de cada módulo

- ☐ Lecturas hechas y ejercicios resueltos.
- ☐ Preguntas de control respondidas por escrito.
- ☐ Pseudocódigo de la discretización en las soluciones.
- ☐ `node -test` en verde, incluidas las pruebas de módulos anteriores.
- ☐ La función pública funciona en computador y celular.
- ☐ Rama fusionada con `main` y subida.
- ☐ Entrada de cierre en la bitácora.